

《人工智能与Python程序设计》—— 多元线性回归



人工智能与Python程序设计 教研组

Numpy数组

- Numpy数组 (ndarray) 是Numpy库中最核心的数据类型
 - 支持对多维数组 (又叫张量 (Tensor)) 的高效存储和访问
 - 其结构如下:

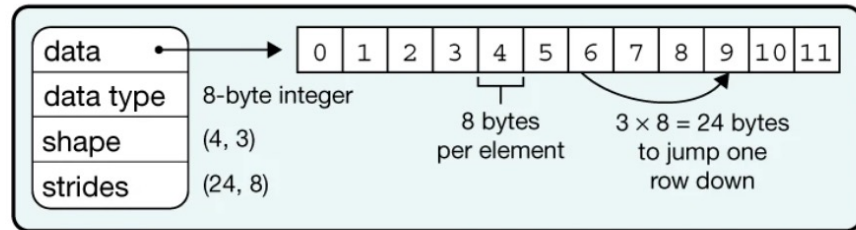
```
In [3]: import numpy as np
x = np.arange(12).reshape((4,3))
print(type(x))
print(x)

<class 'numpy.ndarray'>
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

a Data structure

x =

0	1	2
3	4	5
6	7	8
9	10	11



- 与python内置list不同:

- 所有元素都是一个类型的, 由data type(x.dtype)决定
- 数据保存在一段连续的内存空间中 (与C语言中数组类似)
- 目的: 节省存储空间, 提升访问效率

(from Array programming with NumPy, Nature volume 585, pages357–362(2020))

索引|Numpy数组

- Numpy数组支持比Python内置list更为丰富的索引方式

- 索引一个元素:

`x[1,2] → 5 with scalars`

- 使用切片 (slice) 索引一块子区域:

`x[:,1:] →`

0	1	2
3	4	5
6	7	8
9	10	11

with **slices**

`x[:, ::2] →`

0	1	2
3	4	5
6	7	8
9	10	11

with **slices**
with **steps**

Slices are **start:end:step**,
any of which can be left blank

索引|Numpy数组

- Numpy数组支持比Python内置list更为丰富的索引方式

- 按条件索引:

`x[x > 9] →

10	11
----	----

 with masks`

- 用数组作为数组的索引:

`x[

0	1
---	---

,

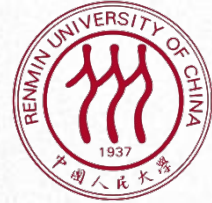
1	2
---	---

] → [x[0,1], x[1,2]] →

1	5
---	---

 with arrays`

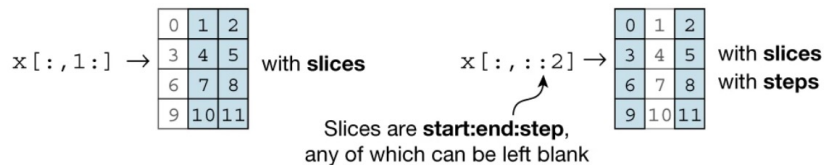
```
# train / test split
shuffled_index = np.random.permutation(data_size)
x = x[shuffled_index]
y = y[shuffled_index]
```

索引|Numpy数组

- 索引是否发生拷贝？
 - 设计思路：尽可能避免拷贝，以节省内存
 - 如果可能，索引操作会返回一个和原数组共享存储的视图 (view)
 - 否则则拷贝数据生成一个新的Numpy数组对象

b Indexing (view)



c Indexing (copy)

$x[1, 2] \rightarrow 5$ with scalars $x[x > 9] \rightarrow$

10	11
----	----

 with masks

$x[\begin{bmatrix} 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 2 \end{bmatrix}] \rightarrow [x[0, 1], x[1, 2]] \rightarrow$

1	5
---	---

 with arrays

$x[\begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 & 0 \end{bmatrix}] \rightarrow x[\begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}] \rightarrow$

4	3
7	6

 with arrays with broadcasting

```
In [7]: x[:, ::2]
```

```
Out[7]: array([[ 0,  2],
               [ 3,  5],
               [ 6,  8],
               [ 9, 11]])
```

```
In [8]: x[:, ::2].strides
```

```
Out[8]: (24, 16)
```

Numpy数组

- 对Numpy数组进行的操作和运算会自动的“向量化”(Vectorization)
 - 分别作用于Numpy数组中的每个元素

d Vectorization

0	1		1	1		1	2
3	4		1	1		4	5
6	7	+	1	1	→	7	8
9	10		1	1		10	11

Numpy数组

- 广播 (Broadcasting)
 - 通过“复制” d次, 将1维变成d维
 - 计算效率高于使用for循环依次计算
 - 以满足向量化计算的要求

e Broadcasting

0		1	2	0	0
3				3	6
6				6	12
9				9	18

Numpy数组

- 归约 (Reduction)
 - 通过求和、求平均数等运算，将d维缩减为1维
 - 以求和函数np.sum为例
 - 可以通过axis可选参数决定按照哪个维度进行计算
 - 默认axis=None，将所有元素求和

```
In [16]: np.sum(x)
```

```
Out[16]: 66
```

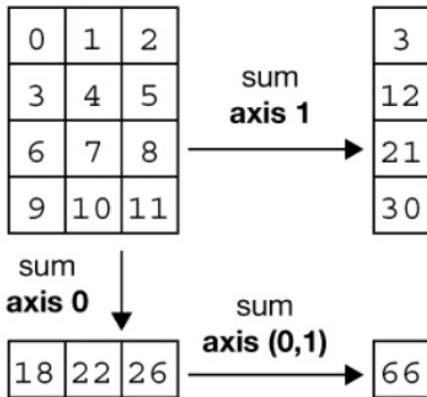
```
In [17]: np.sum(x, axis=0)
```

```
Out[17]: array([18, 22, 26])
```

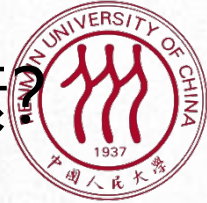
```
In [18]: np.sum(x, axis=1)
```

```
Out[18]: array([ 3, 12, 21, 30])
```

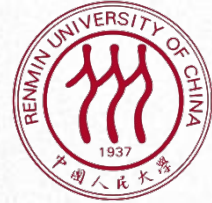
f Reduction



练习题：如何使用索引获得矩阵对角线上的元素？



● 练习题：如何使用普通乘法和broadcasting、reduction机制实现矩阵乘法？





提纲



Python AI 多元线性回归

- ☐ 一元线性回归
- ☐ 使用Matplotlib库进行图表绘制
- ☐ 多元线性回归
-

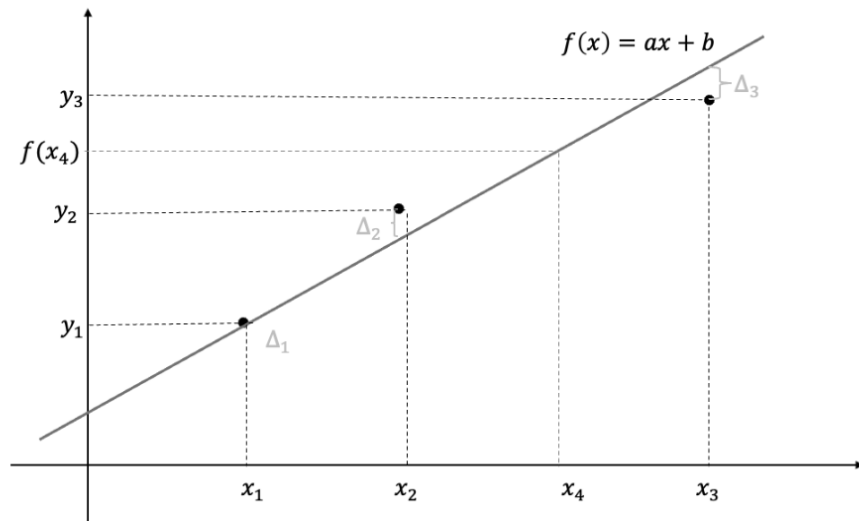


一些常用名词

- 数据
 - 一般为空间中点的集合，每个点以 (x_i, y_i) 的形式出现
 - 一般称 x_i 为“特征”， y_i 为对应的“真实值”或“观测值”
- 任务：找到由 x 得到 y 的方法，即给定 x ，预测 y
- 模型：
 - 猜想 x 到 y 的映射由 $f(x)$ 给出
 - 模型即 $f(x)$ 的具体形式，带有待定参数
- 损失函数：根据已有数据，评价参数质量的指标
- 训练：找到最好的 $f(x)$ 参数的过程，即最小化损失函数的过程
- 预测：给出 x ，计算 $f(x)$ 的过程
- 测试：评价预测结果

一元线性回归

- 在坐标纸上给定三个点的训练集合
 - $(x_1, y_1), (x_2, y_2), (x_3, y_3)$
- 并且规定 $f(x)$ 为线性函数（直线）
 - $f(x) = wx + b$
- 三个点，一条直线
 - 可能无法满足这条直线完美穿越所有的点（训练集合上的误差）
- 训练误差的计算
 - $\frac{1}{3}((f(x_1) - y_1)^2 + (f(x_2) - y_2)^2 + (f(x_3) - y_3)^2)$
- 模型训练要解决的问题：找到使得训练误差最小的参数组合 (w, b)





损失函数

- 使用任何一组参数都可以得到一组预测值 $\hat{y}$ ，需要一个标准来对预测结果进行度量：定量化一个目标函数式度量预测结果的好坏。
- MSE(mean square error)均方误差：线性模型应用于训练样本 x_i ，得到一组预测值 $\hat{y}_i$ ，将预测值和真实值 y_i 之间的平均的平方距离定义为损失函数：

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- N 为样本总数



损失函数

- 将线性预测函数式代入损失函数，将需要求解的参数 w 和 b 看做是损失函数 L 的自变量：

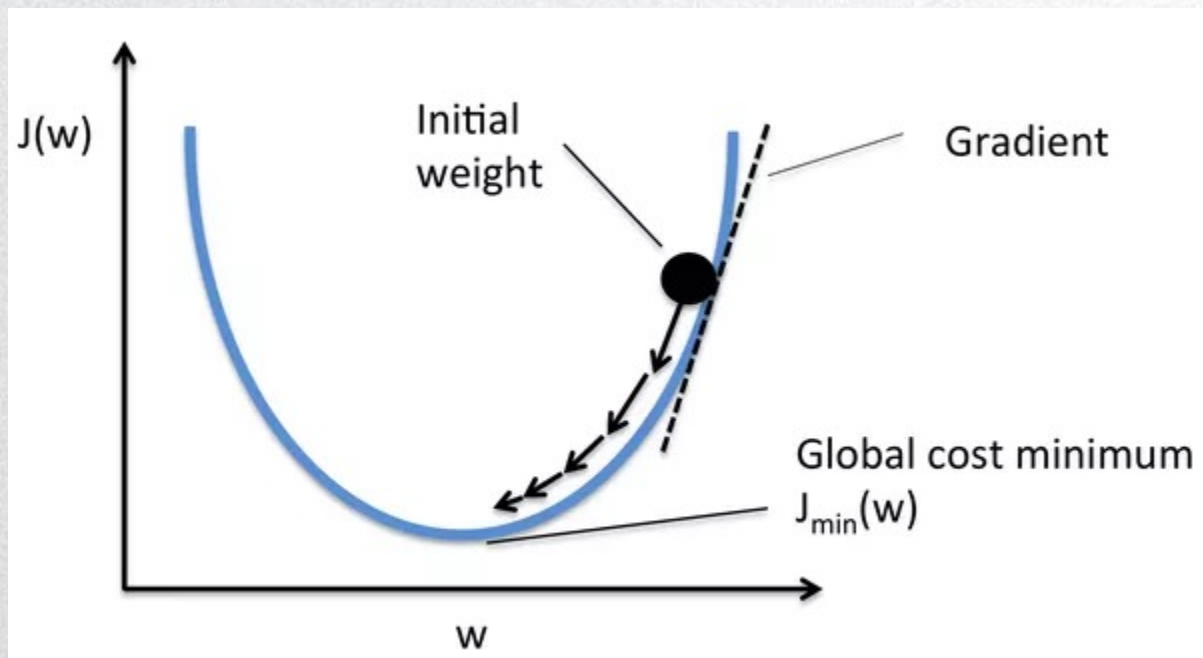
$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

- 通过最小化损失函数求解最优参数：

$$(w^*, b^*) = \arg \min_{w, b} \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

梯度下降法

- 一元线性回归





链式法则



介绍



播报



编辑

链式法则是求复合函数的导数（偏导数）的法则，若 I, J 是直线上的开区间，函数 $f(x)$ 在 I 上有定义 $a \in I$ 处可微，函数 $g(y)$ 在 J 上有定义 $(J \supset f(I))$ ，在 $f(a)$ 处可微，则复合函数 $(g \circ f)(x) = g(f(x))$ 在 a 处可微（ $g \circ f$ 在 I 上有定义），且 $(g \circ f)'(a) = g'(f(a)) f'(a)$ 。若记 $u=g(y), y=f(x)$ ，而 f 在 I 上可微， g 在 J 上可微，则在 I 上任意点 x 有

$$(g \circ f)'(x) = g'(f(x)) f'(x),$$

即 $(g \circ f)'(x) = (g' \circ f)(x) f'(x)$ ，或写出

$$\frac{du}{dx} = \frac{du}{dy} \cdot \frac{dy}{dx},$$

这个结论可推广到任意有限个函数复合到情形，于是复合函数的导数将是构成复合这有限个函数在相应点的导数的乘积，就像锁链一样一环套一环，故称链式法则。

优化

$$(w^*, b^*) = \arg \min_{w, b} \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

- 对 w 求导:

$$\frac{\partial L(w, b)}{\partial w} = \frac{1}{N} \sum_{i=1}^N 2(wx_i + b - y_i)x_i$$

- **向量化**形式:

$$\frac{\partial L(w, b)}{\partial w} = \text{np.mean}(2 * (w\mathbf{x} + b - \mathbf{y}) * \mathbf{x})$$

$\mathbf{x}$, $\mathbf{y}$ 为size为 N 的向量 (1维数组)

优化

$$(w^*, b^*) = \arg \min_{w, b} \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

- 对 b 求导:

$$\frac{\partial L(w, b)}{\partial b} = \frac{2}{N} \sum_{i=1}^N (wx_i + b - y_i) = \frac{2}{N} \sum_{i=1}^N 1 \times (wx_i + b - y_i)$$

- **向量化**形式:

$$\frac{\partial L(w, b)}{\partial w} = \text{np.mean}(2 * (w\mathbf{x} + b - \mathbf{y}))$$

$\mathbf{x}$, $\mathbf{y}$ 为size为 N 的向量 (1维数组)



优化：梯度下降法

- 1. 随机选择一组初始参数(w^0, b^0)
- 2. 对于可微函数，梯度方向时函数增长速度最快的方向，梯度的反方向是函数减少最快的方向。计算损失函数在当前参数点处的梯度。
- 3. 从当前参数点向梯度相反的方向移动，移动步长为：

$$w^{t+1} = w^t - \boxed{lr} \frac{\partial L(w, b)}{\partial w} (w^t)$$

$$b^{t+1} = b^t - \boxed{lr} \frac{\partial L(w, b)}{\partial b} (b^t)$$

学习率

- 4. 循环迭代步骤 3，直到前后两次迭代得到的(w^t, b^t)差值足够小，即参数基本不再变化，说明此时损失函数已经达到局部最小值。
- 5. 输出(w^t, b^t)，即为使得损失函数最小时的参数取值。



示例：梯度下降法

```
class LinearRegression(object):  
  
    def __init__(self, learning_rate=0.01, max_iter=100, seed=None):  
        """  
        一元线性回归类的构造函数：  
        参数 学习率：learning_rate  
        参数 最大迭代次数：max_iter  
        参数 seed：产生随机数的种子  
        从正态分布中采样w和b的初始值  
        """  
        np.random.seed(seed)  
        self.lr = learning_rate  
        self.max_iter = max_iter  
        self.w = np.random.normal(1, 0.1)  
        self.b = np.random.normal(1, 0.1)  
        self.loss_arr = []
```



示例：梯度下降法

```
def fit(self, x, y):  
    """
```

类的方法：训练函数

参数 自变量: x

参数 因变量: y

返回每一次迭代后的损失函数

```
    """
```

```
    for i in range(self.max_iter):
```

```
        self.__train_step(x, y)
```

```
        y_pred = self.predict(x)
```

```
        self.loss_arr.append(self.loss(y, y_pred))
```

```
def __f(self, x, w, b):  
    """
```

类的方法：计算一元线性回归函数在x处的值

```
    """
```

```
    return x * w + b
```

```
def predict(self, x):  
    """
```

类的方法：预测函数

参数：自变量: x

返回：对x的回归值

```
    """
```

```
    y_pred = self.__f(x, self.w, self.b)
```

```
    return y_pred
```

```
def loss(self, y_true, y_pred):  
    """
```

类的方法：计算损失

参数 真实因变量: y_true

参数 预测因变量: y_pred

返回：MSE损失

```
    """
```

```
    return np.mean((y_true - y_pred)**2)
```



示例：梯度下降法

```
def __calc_gradient(self, x, y):  
    """  
    类的方法：分别计算对w和b的梯度  
    """  
    d_w = np.mean(2* (x * self.w + self.b - y) * x)  
    d_b = np.mean(2*(x * self.w + self.b - y))  
    return d_w, d_b  
  
def __train_step(self, x, y):  
    """  
    类的方法：单步迭代，即一次迭代中对梯度进行更新  
    """  
    d_w, d_b = self.__calc_gradient(x, y)  
    self.w = self.w - self.lr * d_w  
    self.b = self.b - self.lr * d_b  
    return self.w, self.b
```



示例：梯度下降法

```
In [36]: import numpy as np
import matplotlib.pyplot as plt

def show_data(x, y, w=None, b=None):
    plt.scatter(x, y, marker='.')
    if w is not None and b is not None:
        plt.plot(x, w*x+b, c='red')
    plt.show()

# data generation
np.random.seed(272)
data_size = 100
x = np.random.uniform(low=1.0, high=10.0, size=data_size)
y = x * 20 + 10 + np.random.normal(loc=0.0, scale=10.0, size=data_size)
```




示例：梯度下降法

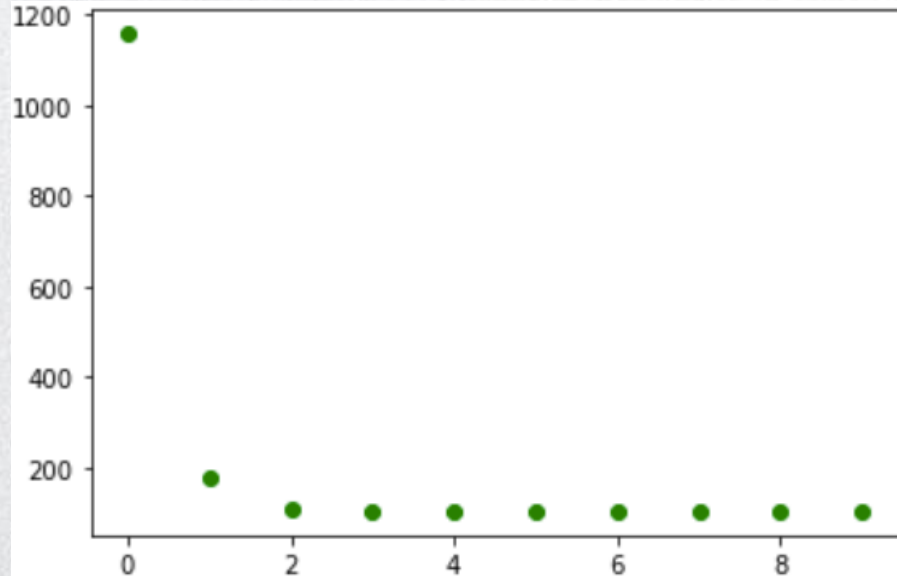
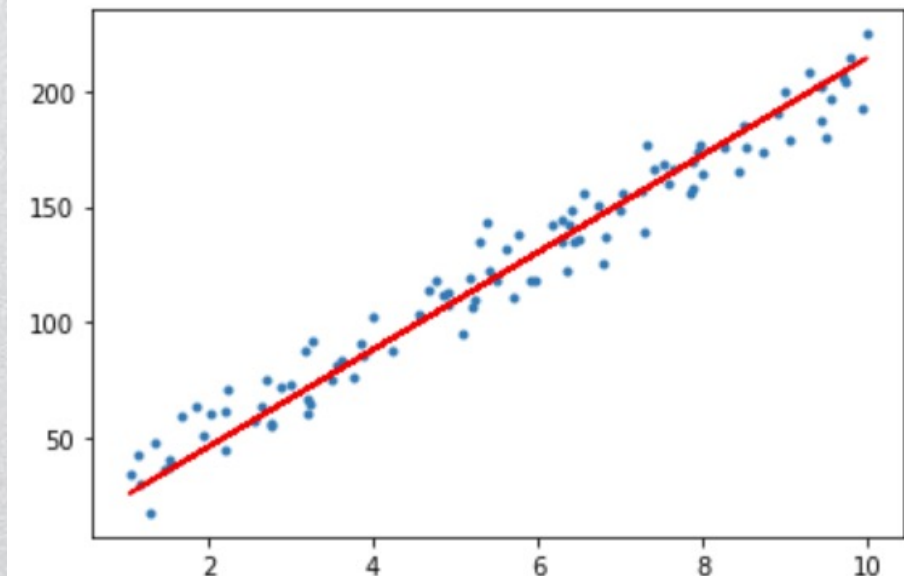
```
# train / test split
shuffled_index = np.random.permutation(data_size)
x = x[shuffled_index]
y = y[shuffled_index]
split_index = int(data_size * 0.7)
x_train = x[:split_index]
y_train = y[:split_index]
x_test = x[split_index:]
y_test = y[split_index:]

# train the liner regression model
regr = LinearRegression(learning_rate=0.01, max_iter=10, seed=314)
regr.fit(x_train, y_train)
print('w: \t{:.3}'.format(regr.w))
print('b: \t{:.3}'.format(regr.b))
show_data(x, y, regr.w, regr.b)

# plot the evolution of cost
plt.scatter(np.arange(len(regr.loss_arr)), regr.loss_arr, marker='o', c='green')
plt.show()
```

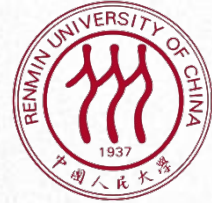
示例：梯度下降法

w: 21.0
b: 4.41





思考



- 多元线性回归：
 - 自变量 x 是向量，因变量 y 是数字
 - 自变量 x 是向量，因变量 y 也是向量
- 怎样用梯度下降法训练多元线性回归模型？



提纲



Python AI 多元线性回归

- □ 一元线性回归
- □ 使用Matplotlib库进行图表绘制
- □ 多元线性回归
-



matplotlib库简介

- Matplotlib是一个功能强大的Python绘图和可视化库
 - 我们主要介绍其中pyplot子库的功能
- 安装方式
 - pip install matplotlib
- 官方主页：
 - <https://matplotlib.org/>

matplotlib



使用Matplotlib库进行图表绘制

- 绘图环境设置:

```
[58]: # 导入pyplot子库
import matplotlib.pyplot as plt

# 对绘图环境进行设置, 选择使用中文字体
import matplotlib
matplotlib.rcParams['font.family']='SimHei'
matplotlib.rcParams['font.sans-serif']=['SimHei']

# 使jupyter notebook能直接显示matplotlib绘制的结果
%matplotlib inline
```



使用Matplotlib库进行图表绘制

- 以一元线性回归和逻辑回归问题为例，介绍如何使用matplotlib绘制：
 - 散点图
 - 折线图
 - 直方图
 - 二维直方图
 - 等高线图



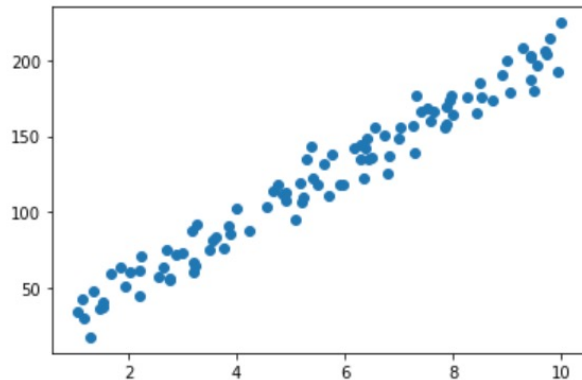
使用Matplotlib库进行图表绘制

- 散点图

- 展示(x, y)数据点的分布，使用plt.scatter(x, y)函数绘制
 - x, y是两个长度相等的list，或shape为(n,)的一维ndarray，表示点的坐标
 - 可选参数：
 - maker: 点的样式（参见：https://matplotlib.org/stable/api/markers_api.html#module-matplotlib.markers）
 - c: 点的颜色
- 以一元线性回归问题中生成的数据为例：

```
[5]: import numpy as np

# 生成数据
np.random.seed(272)
data_size = 100
# x在1到10间均匀分布
x = np.random.uniform(low=1.0, high=10.0, size=data_size)
# y = x * 20 + 10 + 随机噪声
y = x * 20 + 10 + np.random.normal(loc=0.0, scale=10.0, size=data_size)
plt.scatter(x, y)
plt.show()
```

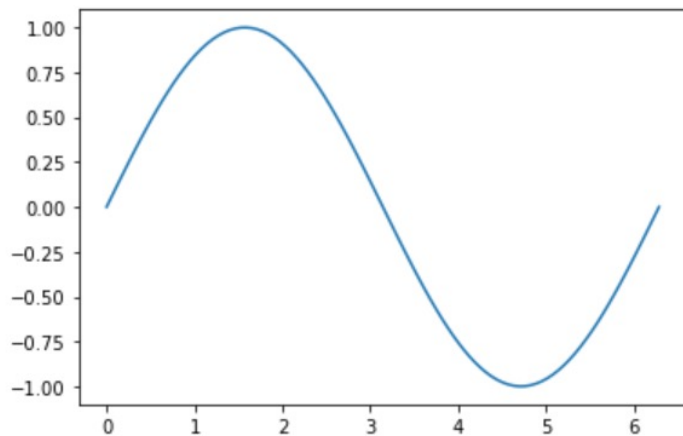




使用Matplotlib库进行图表绘制

- 折线图：
 - 展示y随x的变化情况，使用`plt.plot(x, y, [fmt])`函数绘制
 - x, y是两个长度相等的list，或shape为(n,)的一维ndarray，表示点的坐标
 - 同样可以用额外的可选参数 (fmt) 控制绘制的样式
 - 请在jupyter notebook下执行`plt.plot?`查看相关帮助
 - 例：绘制Sin函数在 $[0, 2\pi]$ 范围内的图像

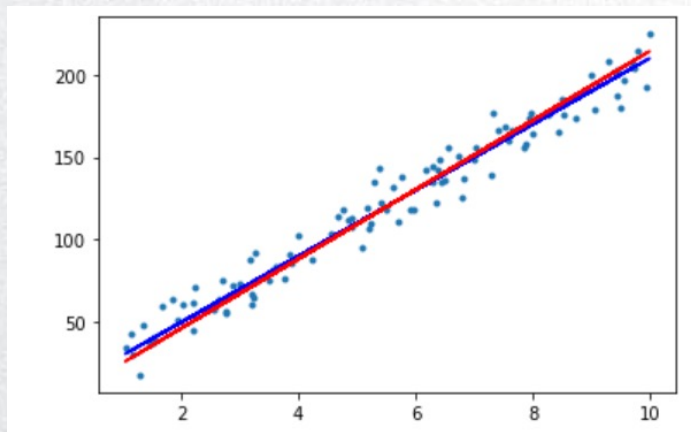
```
[12]: # 绘制折线图
x = np.linspace(0, 2*np.pi, num=100)
y = np.sin(x)
plt.plot(x, y)
plt.show()
```



使用Matplotlib库进行图表绘制

- 折线图：
 - 展示y随x的变化情况，使用plt.plot(x, y)函数绘制
 - 例：展示一元线性回归模型在训练集上学习到的直线方程
 - $y = f(x) = wx + b$
 - 并与数据点及生成数据的数据方程进行比较
 - 在执行plt.show()之前执行多次绘图函数

```
[20]: # 绘制数据点
plt.scatter(x, y, marker='.')
# 用蓝线绘制生成数据时的方程
plt.plot(x, x * 20 + 10, 'b')
# 用红线绘制在训练集上拟合得到的直线方程
plt.plot(x, x * regr.w + regr.b, 'r')
plt.show()
```





使用Matplotlib库进行图表绘制

- 直方图：
 - 展示一维数据的分布，使用`plt.hist(x)`函数绘制
 - 将数据按照大小放到`n`个桶里，用柱状图的展示每个桶里的数据点个数
 - `x`: 包含需要展示的数据的`list`或者一维`ndarray`
 - 部分可选参数：
 - `bins`: 桶的个数（或者以序列形式给出的桶的边界）
 - `color`: 直方图的颜色

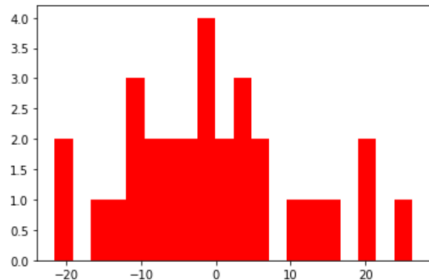
使用Matplotlib库进行图表绘制

- 直方图：
 - 例：展示一元线性回归模型的测试误差的分布
 - $error_i = \hat{y}_i - y_i = wx_i + b - y_i$

```
[68]: # 生成数据
# 数据集大小为100
x, y, x_train, y_train, x_test, y_test = generate_data(100)

# 训练模型
regr = LinearRegression(learning_rate=0.01, max_iter=10, seed=314)
regr.fit(x_train, y_train)

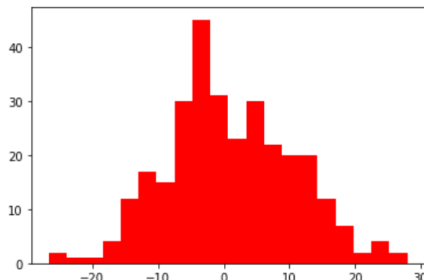
# 测试误差
test_error = y_test - regr.predict(x_test)
plt.hist(test_error, color='r', bins=20)
plt.show()
```



```
[82]: # 生成数据
# 数据集大小为1000
x, y, x_train, y_train, x_test, y_test = generate_data(1000)

# 训练模型
regr = LinearRegression(learning_rate=0.01, max_iter=10, seed=314)
regr.fit(x_train, y_train)

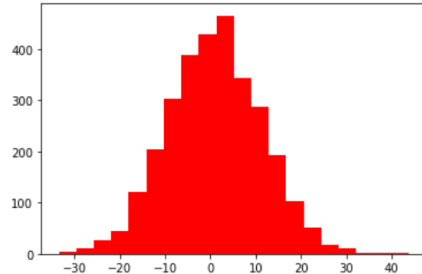
# 测试误差
test_error = y_test - regr.predict(x_test)
plt.hist(test_error, color='r', bins=20)
plt.show()
```



```
[83]: # 生成数据
# 数据集大小为10000
x, y, x_train, y_train, x_test, y_test = generate_data(10000)

# 训练模型
regr = LinearRegression(learning_rate=0.01, max_iter=10, seed=314)
regr.fit(x_train, y_train)

# 测试误差
test_error = y_test - regr.predict(x_test)
plt.hist(test_error, color='r', bins=20)
plt.show()
```



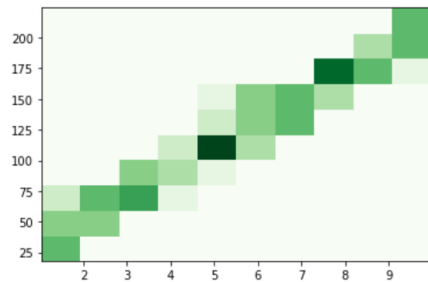
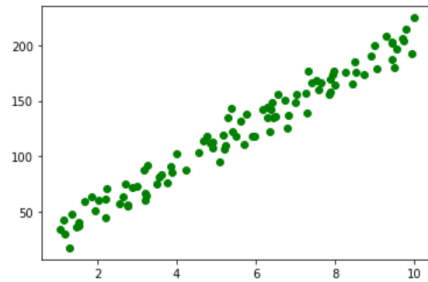
使用Matplotlib库进行图表绘制

- 二维直方图（热度图）：
 - 展示**二维**数据的分布，使用plt.hist2d(x, y)函数绘制
 - 将数据按照(x, y)坐标放到n*n个桶里，用**颜色**每个桶里的数据点个数
 - x, y: 两个长度相等的list，或shape为(n,)的一维ndarray，表示点的坐标
 - 可选参数：
 - cmap: 使用不同的颜色来绘制分布

```
[93]: # 生成数据
x, y, x_train, y_train, x_test, y_test = generate_data(100, noise_scale=10.0)

# 散点图
plt.scatter(x, y, c='g')
plt.show()

# 二维直方图
plt.hist2d(x, y, cmap='Greens')
plt.show()
```





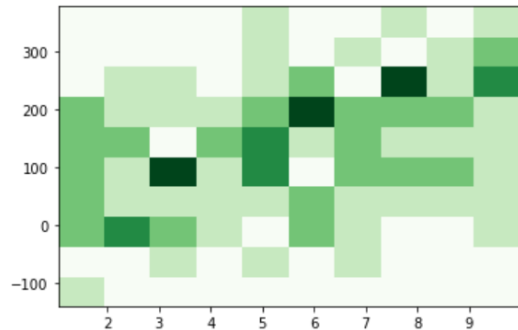
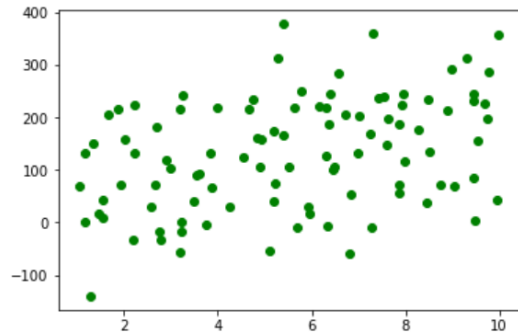
使用Matplotlib库进行图表绘制

- 二维直方图（热度图）：
 - 展示**二维**数据的分布，使用plt.hist2d(x, y)函数绘制
 - 改变数据的噪声

```
[94]: # 生成数据
x, y, x_train, y_train, x_test, y_test = generate_data(100, noise_scale=100.0)

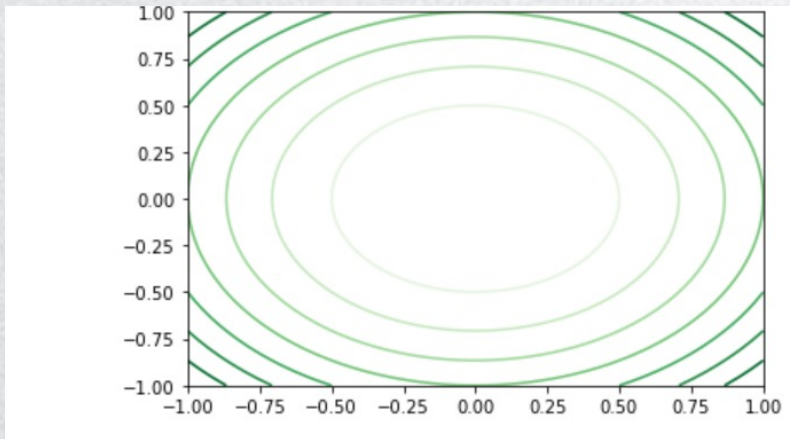
# 散点图
plt.scatter(x, y, c='g')
plt.show()

# 二维直方图
plt.hist2d(x, y, cmap='Greens')
plt.show()
```



使用Matplotlib库进行图表绘制

- 等高线图：
 - 展示三维数据的分布，使用`plt.contour()`函数绘制
 - 用等高线的方式，绘制出 $z = f(x, y)$ 的图像
 - 例：展示 $z = x^2 + y^2$



使用Matplotlib库进行图表绘制

- 等高线图:

- 例: 展示 $z = x^2 + y^2$

- 第一步: 生成 (x, y) 坐标网格

```
[113]: # 例: 绘制 $z=x^2+y^2$   
# 给定x和y的范围  
x = np.linspace(-1, 1, num=5)  
y = np.linspace(-1, 1, num=5)  
# 使用np.meshgrid函数生成一个网格  
x, y = np.meshgrid(x, y)  
print(x)  
print(y)
```

```
[[ -1.  -0.5  0.   0.5  1. ]  
 [ -1.  -0.5  0.   0.5  1. ]  
 [ -1.  -0.5  0.   0.5  1. ]  
 [ -1.  -0.5  0.   0.5  1. ]  
 [ -1.  -0.5  0.   0.5  1. ]]  
[[ -1.  -1.  -1.  -1.  -1. ]  
 [-0.5 -0.5 -0.5 -0.5 -0.5]  
 [ 0.   0.   0.   0.   0. ]  
 [ 0.5  0.5  0.5  0.5  0.5]  
 [ 1.   1.   1.   1.   1. ]]
```


使用Matplotlib库进行图表绘制

- 等高线图:

- 例: 展示 $z = x^2 + y^2$

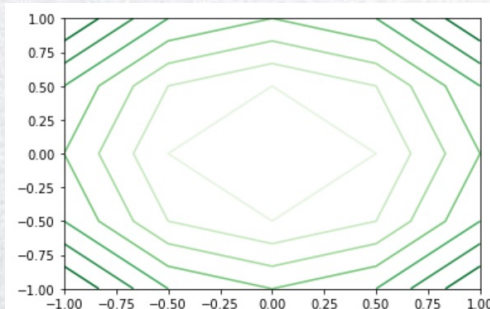
- 第二步: 计算 z 在 (x, y) 坐标网格上每一点的值

```
[115]: # 计算z在网格上每一点的值
z = x**2 + y**2
print(z)

[[2.   1.25 1.   1.25 2.   ]
 [1.25 0.5  0.25 0.5  1.25]
 [1.   0.25 0.   0.25 1.   ]
 [1.25 0.5  0.25 0.5  1.25]
 [2.   1.25 1.   1.25 2.   ]]
```

- 第三步: 绘制等高线图
 - 同样可以用cmap调整颜色

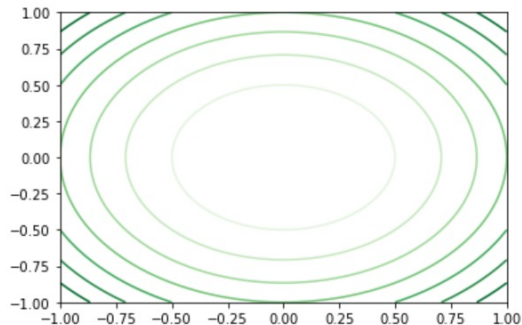
```
[117]: plt.contour(x, y, z, cmap="Greens")
plt.show()
```



使用Matplotlib库进行图表绘制

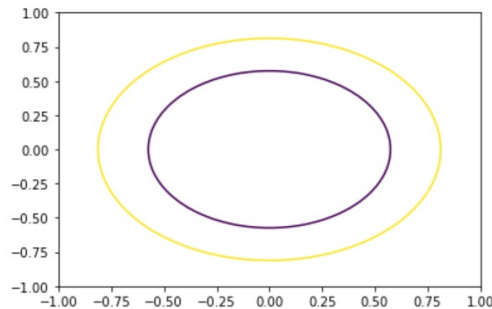
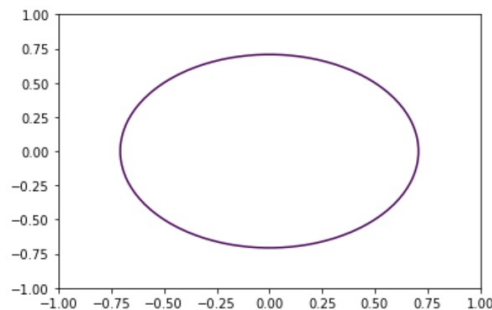
- 等高线图：
 - 例：展示 $z = x^2 + y^2$
 - 使用更密的网格，使图像更平滑
 - 使用可选参数
 - 制定绘制所需的等高线

```
[119]: # 用更多的点，使函数更为平滑
x = np.linspace(-1, 1, num=50)
y = np.linspace(-1, 1, num=50)
x, y = np.meshgrid(x, y)
z = x**2 + y**2
plt.contour(x, y, z, cmap="Greens")
plt.show()
```



```
[123]: # 只保留z=0.5一条等高线
plt.contour(x, y, z, [0.5])
plt.show()

# 保留z=[0.33, 0.66]两条等高线
plt.contour(x, y, z, [0.33, 0.66])
plt.show()
```



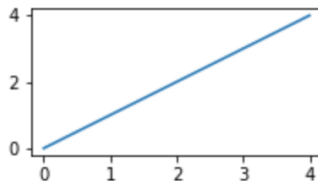
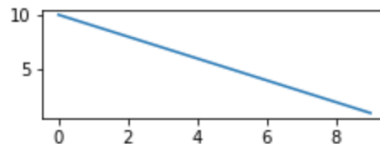
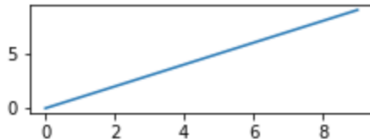
使用Matplotlib库进行图表绘制

- 绘制多个子图:

```
# 绘图环境的设置
plt.figure(figsize=(8,4)) # 设置画布大小
plt.subplot(321) # 在一个三行两列的绘图区域中选择子区域1开始绘图
plt.plot(range(10), range(10)) # x=0..9, y=0..9

plt.subplot(324) # 在一个三行两列的绘图区域中选择子区域4开始绘图
plt.plot(range(10), range(10,-1))

# 直接在画布上按照[左边缘位置, 下边缘位置, 宽度, 高度]选择绘图区域
plt.axes([0.1, 0.1, 0.3, 0.3])
plt.plot(range(5), range(5))
plt.show()
```





使用Matplotlib库进行图表绘制

- 保存绘制的图像：
 - 将plt.show()换成plt.savefig(文件名)
 - 用文件名中的扩展名 (.xxx) 指定保存的图片文件的格式
 - 支持的格式：
 - 位图：
 - 由二维排布的像素组成的图像
 - 像素放大后会有明显的锯齿
 - JPG (有损压缩)
 - PNG (无损)
 - 矢量图：
 - 用点、直线或者多边形等基于数学方程的几何图元表示图像
 - 可平滑放大
 - SVG
 - PDF



小结

- matplotlib是一个功能强大的python绘图库
- 主要学习了基于pyplot子库绘制不同类型的图表
- 可以将绘制好的图表保存成位图或者矢量图



一元线性回归的可视化

- 将线性预测函数式代入损失函数，将需要求解的参数 w 和 b 看做是损失函数 L 的自变量：

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

- 通过梯度下降最小化损失函数求解最优参数：

$$(w^*, b^*) = \arg \min_{w, b} \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

- 使用等高线图可视化 $L(w, b)$ ，用折线图展示更新过程

一元线性回归的可视化

$$(w^*, b^*) = \arg \min_{w, b} \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

- 对 w 求导:

$$\frac{\partial L(w, b)}{\partial w} = \frac{1}{N} \sum_{i=1}^N 2(wx_i + b - y_i)x_i$$

- 矩阵形式:

$$\frac{\partial L(w, b)}{\partial w} = \frac{2}{N} \mathbf{x}^T (\mathbf{w}\mathbf{x} + b\mathbf{1} - \mathbf{y})$$

$\mathbf{x}$ 、 $\mathbf{1}$ 、 $\mathbf{y}$ 为 $N \times 1$ 的向量 (列向量)

一元线性回归的可视化

$$(w^*, b^*) = \arg \min_{w, b} \frac{1}{N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

- 对 b 求导:

$$\frac{\partial L(w, b)}{\partial b} = \frac{2}{N} \sum_{i=1}^N (wx_i + b - y_i) = \frac{2}{N} \sum_{i=1}^N 1 \times (wx_i + b - y_i)$$

- 矩阵形式:

$$\frac{\partial L(w, b)}{\partial b} = \frac{2}{N} \mathbf{1}^T (w\mathbf{x} + b\mathbf{1} - \mathbf{y})$$

$\mathbf{x}$ 、 $\mathbf{1}$ 、 $\mathbf{y}$ 为 $N \times 1$ 的向量 (列向量)



如何绘制 $L(w, b)$ 的等高线图?

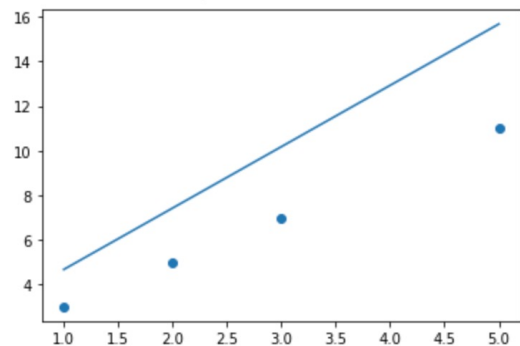
- 先生成 (w, b) 的坐标网格 (meshgrid)
- 对网格中每一个点 (j, k) 计算 $L(w_{j,k}, b_{j,k}; \mathbf{x}, \mathbf{y})$

$$L(w_{j,k}, b_{j,k}; \mathbf{x}, \mathbf{y}) = \frac{1}{N} \sum_{i=1}^N (w_{j,k} \cdot x_i + b_{j,k} - y_i)^2$$

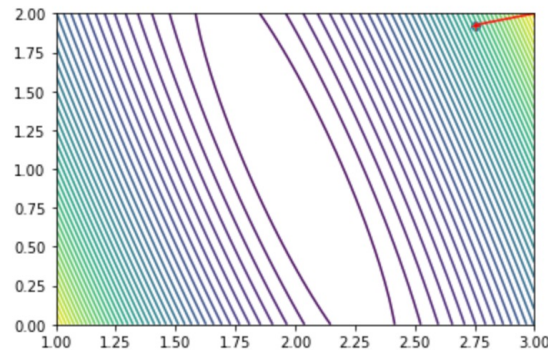
- 充分利用向量化和广播机制

一元线性回归的可视化

#iter: 1: $y=2.75*x+1.925$

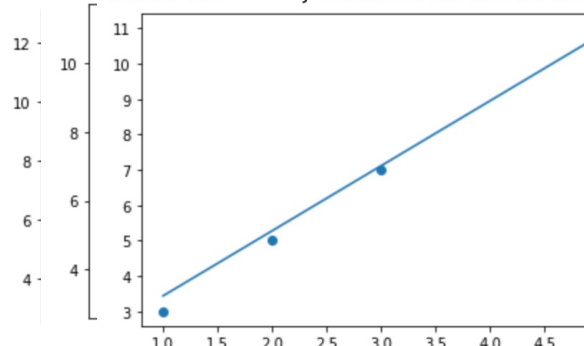


current loss: 10.155625000000004

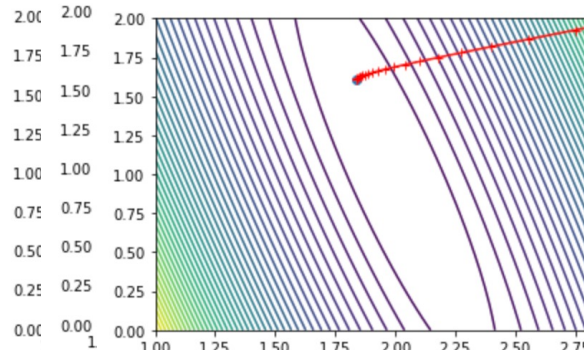


#it #ite

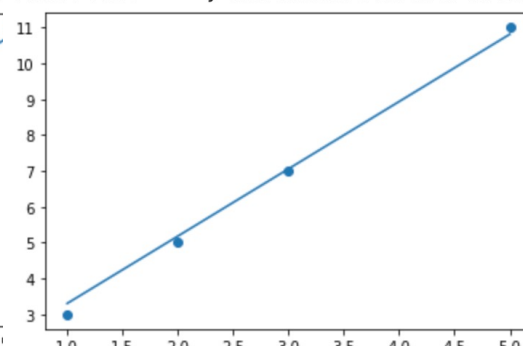
#iter: 20: $y=1.836147046670237*x+1.60$



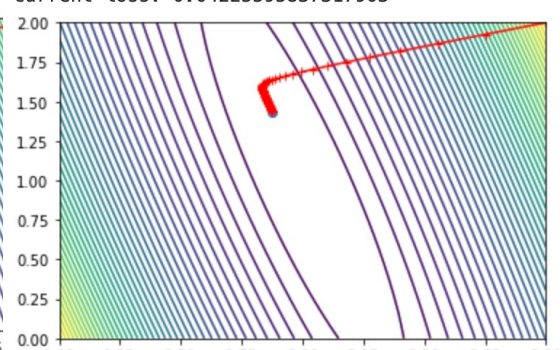
current loss: 0.08343937315641942



#iter: 100: $y=1.8750351630296282*x+1.433628286804934$



current loss: 0.04225595837317903





提纲

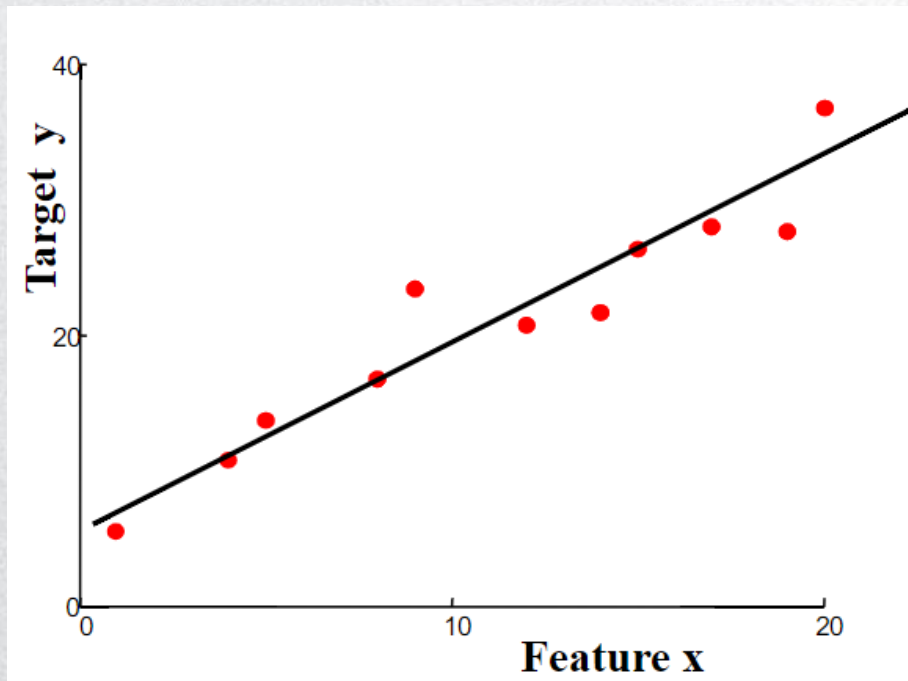


Python AI 多元线性回归

- □ 一元线性回归
- □ 使用Matplotlib库进行图表绘制
- □ 多元线性回归
-

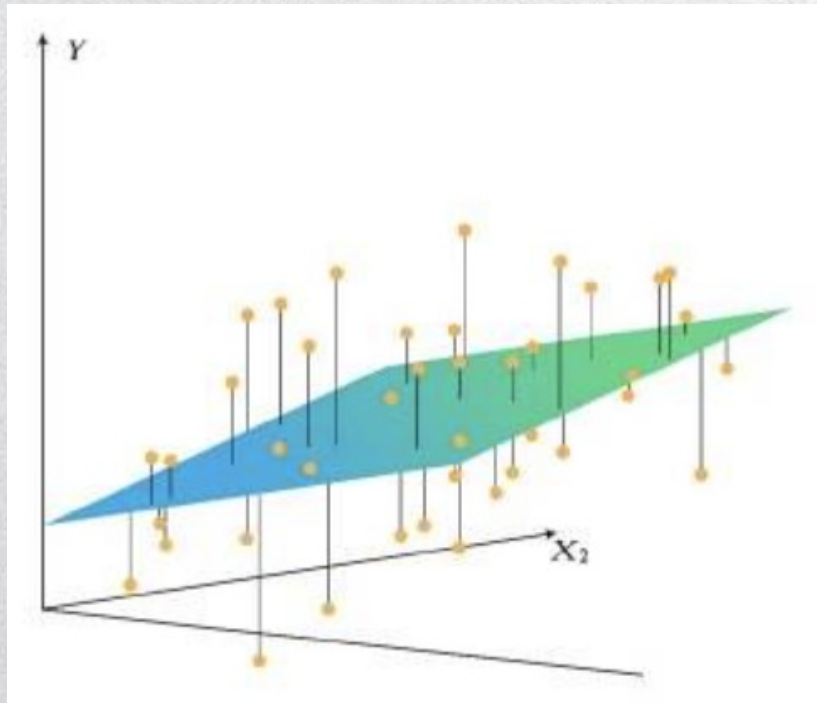
一元线性回归

- 只有一个自变量（特征 x 的维度为1，因此 w 的维度也是1）



多元线性回归

- 有多个自变量（特征 x 的维度大于1，对应的 w 的维度也大于1）



$$\mathbf{x}_i = (x_{i,1}, x_{i,2}, \dots, x_{i,j}, \dots, x_{i,d})$$

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_j \\ \dots \\ w_d \end{pmatrix}$$



多元线性回归

- 假设目标值与特征之间线性相关，即因变量与自变量满足一个多元一次方程。
- 训练集 $D=\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$
- 将特征 $\mathbf{x}$ 和对应的目标 y 建模为多元一次方程：

$$\hat{y} = \mathbf{x} \cdot \mathbf{w} + b$$

其中， $\mathbf{w}$ 和 b 为模型的参数。

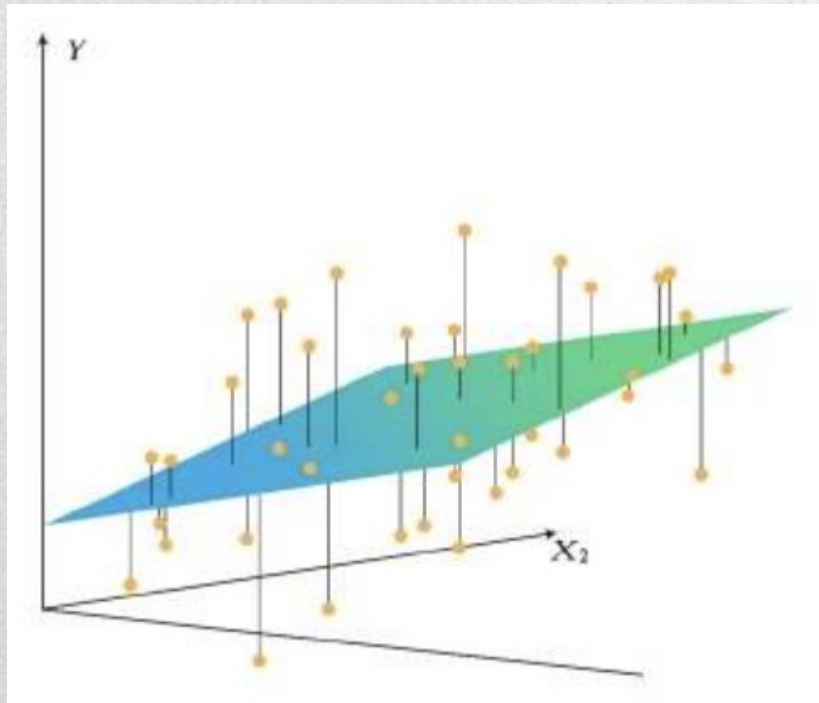
- 为了简化符号表示，在 $\mathbf{x}$ 后面加上一维值恒为1的特征， $\mathbf{x} \leftarrow [\mathbf{x}, 1]$ ，同时 $\mathbf{w}$ 增加一维，则可以把 b 融入 $\mathbf{w}$

$$\hat{y} = \mathbf{x} \cdot \mathbf{w}$$

- 为了学习这两个参数，根据已知的训练样本点（自变量 $\mathbf{x}$ 和因变量 y 都是已知的）拟合这个多元一次方程，求解参数。

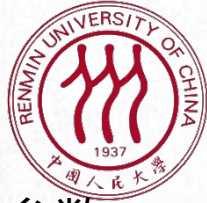
多元线性回归

- 有多个自变量（特征 x 的维度大于1，对应的 w 的维度也大于1）



$$\mathbf{x}_i = (x_{i,1}, x_{i,2}, \dots, x_{i,j}, \dots, x_{i,d}, \mathbf{1})$$

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_j \\ \dots \\ w_d \\ w_{d+1} \end{pmatrix}$$



损失函数

- 将线性预测函数式代入MSE损失函数 $L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$ ，将需要求解的参数 w 看做是损失函数 L 的自变量：

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w} - y_i)^2$$

- 通过最小化损失函数求解最优参数：

$$(\mathbf{w}^*) = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w} - y_i)^2$$

求梯度

$$(\mathbf{w}^*) = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w} - y_i)^2$$

- 分别对参数 $\mathbf{w}$ 求导:

$$\frac{\partial L}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N x_{i,1} (\mathbf{x}_i \cdot \mathbf{w} - y_i)$$

$$\frac{\partial L}{\partial w_2} = \frac{2}{N} \sum_{i=1}^N x_{i,2} (\mathbf{x}_i \cdot \mathbf{w} - y_i)$$

.....

$$\frac{\partial L}{\partial w_{d+1}} = \frac{2}{N} \sum_{i=1}^N x_{i,d+1} (\mathbf{x}_i \cdot \mathbf{w} - y_i)$$

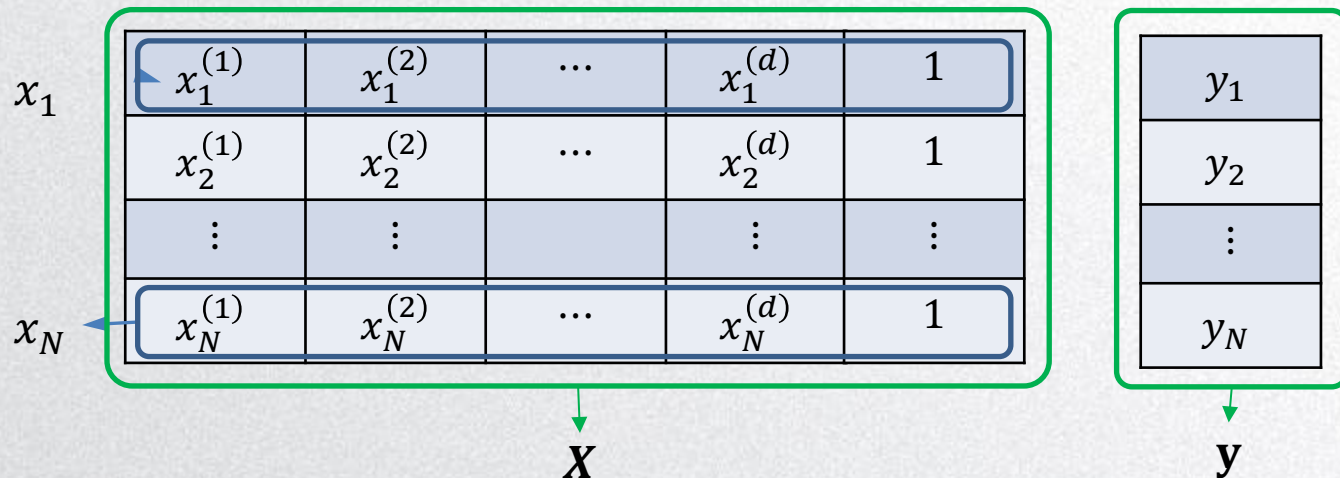
求梯度

$$(\mathbf{w}^*) = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i \cdot \mathbf{w} - y_i)^2$$

- 分别对参数 $\mathbf{w}$ 求导:

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \dots \\ \frac{\partial L}{\partial w_{d+1}} \end{pmatrix} = \begin{pmatrix} \frac{2}{N} \sum_{i=1}^N x_{i,1} (\mathbf{x}_i \cdot \mathbf{w} - y_i) \\ \frac{2}{N} \sum_{i=1}^N x_{i,2} (\mathbf{x}_i \cdot \mathbf{w} - y_i) \\ \dots \\ \frac{2}{N} \sum_{i=1}^N x_{i,d+1} (\mathbf{x}_i \cdot \mathbf{w} - y_i) \end{pmatrix} = \frac{2}{N} \sum_{i=1}^N \mathbf{x}_i^T (\mathbf{x}_i \cdot \mathbf{w} - y_i)$$

梯度的矩阵形式



$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{2}{N} \sum_{i=1}^N \mathbf{x}_i^T (\mathbf{x}_i \cdot \mathbf{w} - y_i), \text{ 其中 } \mathbf{x}_i \text{ 为行向量}$$

- 矩阵形式:

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{2}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y})$$

注意: 此处 $\mathbf{X}$ 的维度是 $N \times (d + 1)$, $\mathbf{w}$ 维度为 $(d + 1) \times 1$



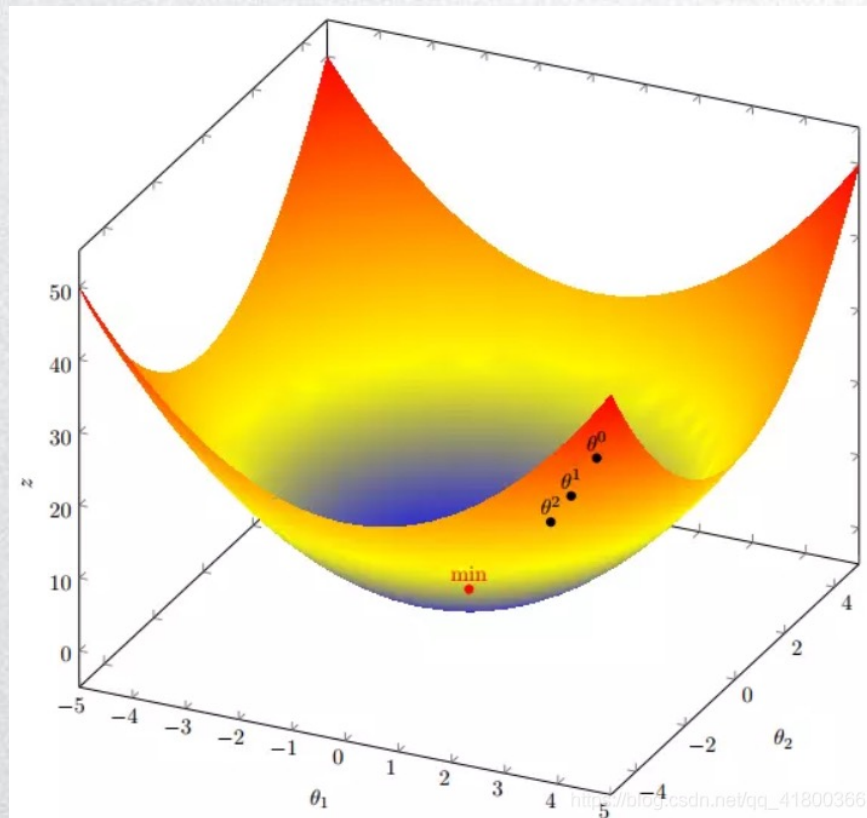
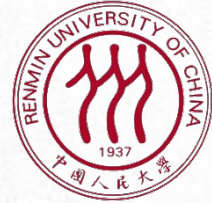
优化：梯度下降法

- 1. 随机选择一组初始参数($\mathbf{w}^0$)
- 2. 对于可微函数，梯度方向时函数增长速度最快的方向，梯度的反方向是函数减少最快的方向。计算损失函数在当前参数点处的梯度。
- 3. 从当前参数点向梯度相反的方向移动，移动步长为：

$$\mathbf{w}^{t+1} = \mathbf{w}^t - lr \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}(\mathbf{w}^t)$$

- 4. 循环迭代步骤 3，直到前后两次迭代得到的($\mathbf{w}^t$)差值足够小，即参数基本不再变化，说明此时损失函数已经达到局部最小值。
- 5. 输出($\mathbf{w}^t$)，即为使得损失函数最小时的参数取值。

梯度下降法





示例：梯度下降法（类的定义和初始化）

```
class MultiLinearRegression(object):  
    def __init__(self, dim_in, learning_rate=0.01, max_iter=100, seed=None):  
        """  
        一元线性回归类的构造函数：  
        参数 学习率: learning_rate  
        参数 最大迭代次数: max_iter  
        参数 seed: 产生随机数的种子  
        从正态分布中采样w的初始值  
        """  
        np.random.seed(seed)  
        self.lr = learning_rate  
        self.max_iter = max_iter  
        self.w = np.random.normal(1, 0.1, [dim_in+1, 1]) # w 的维度为输入维度+1  
        self.loss_arr = []
```



示例：梯度下降法

```
def fit(self, x, y):
```

```
    """
```

```
    类的方法：训练函数
```

```
    参数 自变量: x
```

```
    参数 因变量: y
```

```
    返回每一次迭代后的损失函数
```

```
    """
```

```
    # 首先在x矩阵后面增加一列1
```

```
    x = np.hstack([x, np.ones((x.shape[0], 1))])
```

```
    for i in range(self.max_iter):
```

```
        self.__train_step(x, y)
```

```
        y_pred = self.predict(x)
```

```
        self.loss_arr.append(self.loss(y, y_pred))
```

```
def __train_step(self, x, y):
```

```
    """
```

```
    类的方法：单步迭代，即一次迭代中对梯度进行更新
```

```
    """
```

```
    d_w = self.__calc_gradient(x, y)
```

```
    self.w = self.w - self.lr * d_w
```

```
    return self.w
```

```
def loss(self, y_true, y_pred):
```

```
    """
```

```
    类的方法：计算损失
```

```
    参数 真实因变量: y_true
```

```
    参数 预测因变量: y_pred
```

```
    返回: MSE 损失
```

```
    """
```

```
    return np.mean((y_true - y_pred) ** 2)
```

```
def __calc_gradient(self, X, y):
```

```
    """
```

```
    类的方法：分别计算对w和b的梯度
```

```
    """
```

```
    N = X.shape[0]
```

```
    #diff = (X.dot(self.w) - y)
```

```
    diff = np.matmul(X, self.w) - y
```

```
    #print(type(X.T))
```

```
    grad = np.matmul(X.T, diff) # X.T 转置 x.transpose()
```

```
    d_w = (2 * grad) / N
```

```
    return d_w
```

```
def __f(self, X, w):
```

```
    """
```

```
    类的方法：计算一元线性回归函数在x处的值
```

```
    """
```

```
    return np.matmul(X, w)
```

```
def predict(self, X):
```

```
    """
```

```
    类的方法：预测函数
```

```
    参数：自变量: x
```

```
    返回：对x的回归值
```

```
    """
```

```
    y_prd = self.__f(X, self.w)
```

```
    return y_prd
```

示例：梯度下降法（数据准备）

29

```
# data generation
np.random.seed(272)
data_size = 100
dim_in = 3
dim_out = 1
x = np.random.uniform(low=1.0, high=10.0, size=[data_size, dim_in])
map_true = np.array([[1.5], [-5.], [3.]])
y = x.dot(map_true) + np.random.normal(loc=0.0, scale=10.0, size=[data_size, dim_out])

# train / test split
shuffled_index = np.random.permutation(data_size)
x = x[shuffled_index, :]
y = y[shuffled_index, :]
split_index = int(data_size * 0.7)
x_train = x[:split_index, :]
y_train = y[:split_index, :]
x_test = x[split_index:, :]
y_test = y[split_index:, :]
```




示例：梯度下降法（训练模型）

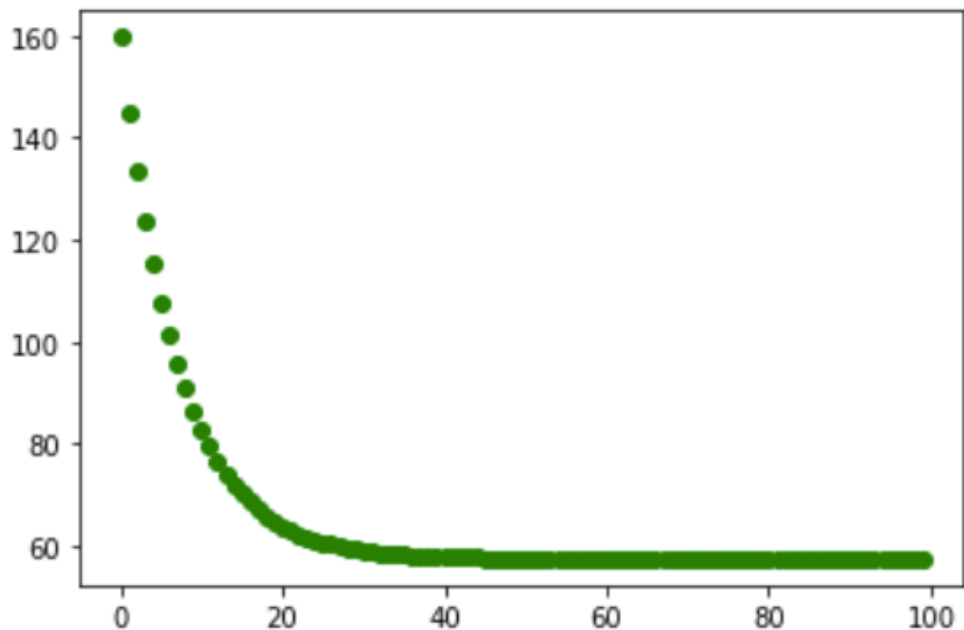
```
# train the liner regression model
regr = LinearRegressionMulti(dim_in, learning_rate=0.01, max_iter=100, seed=0)
regr.fit(x_train, y_train)
print(regr.w)

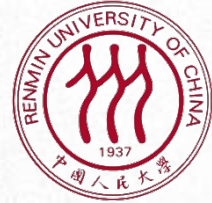
x_test_aug = np.hstack([x_test, np.ones((x_test.shape[0], 1))])
y_pred = regr.predict(x_test_aug)
res = y_pred - y_test

# plot the evolution of cost
plt.scatter(np.arange(len(regr.loss_arr)), regr.loss_arr, marker='o', c='green')
plt.show()
```

示例：梯度下降法

```
[[ 0.53989083]  
 [-4.89837128]  
 [ 3.30861944]  
 [ 0.78811499]]
```





谢谢！